# 10 — Interpretabilidad del Modelo

## Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook explica el **modelo final seleccionado en el notebook 08: XGBoost** con selector `SelectKBest` (Mutual Information, 2000 features). No evalúa rendimiento: las métricas siguen siendo las OOF del notebook 09.

### Alcance

- SHAP con `TreeExplainer` sobre el XGBoost real (multiclase, 3 clases).
- Importancia global y por clase ( `health` , `sick` , `tb` ).
- Importancia por familia de características.
- Explicaciones individuales OOF con el modelo del fold correspondiente.
- Atribución espacial por oclusión y solape con las cajas de lesión.

Los valores SHAP son atribuciones al **output del modelo** (margen/log-odds para `multi:softprob` ), no probabilidades. El eje de clase sigue `clf.classes_` = `[0, 1, 2]` .

# 0. Configuración del entorno

In [6]:
```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   SHAP TreeExplainer: https://shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html
#   XGBoost: https://xgboost.readthedocs.io/en/stable/

import sys, os, warnings, json, tempfile
warnings.filterwarnings('ignore')

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
import cv2
import joblib
import shap

from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold
from sklearn.utils.class_weight import compute_sample_weight
from xgboost import XGBClassifier
from sklearn.metrics import balanced_accuracy_score, recall_score

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.preprocessing import preprocess
from src.features import extract_features
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

```
CLASS_NAMES = ['health', 'sick', 'tb']
DATA_DIR    = Path('..') / 'data'
MODELS_DIR  = Path('..') / 'models'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
print(f'SHAP {shap.__version__}')
```

```
Entorno configurado correctamente.
SHAP 0.54.1
```

# 1. Carga y verificación de artefactos

In [7]:

```
# Se comprueban el modelo final, el manifiesto, la selección del 06 y las predicciones OOF.

ARCHIVOS = [
    MODELS_DIR / '08_xgboost.joblib',
    MODELS_DIR / '08_manifest.json',
    DATA_DIR / '06_seleccion.json',
    DATA_DIR / '06_selected_feature_names.json',
    DATA_DIR / 'X_full.npy',
    DATA_DIR / 'y_full.npy',
    DATA_DIR / '08_oof_predicciones.csv',
    DATA_DIR / '08_oof_scores.npz',
    DATA_DIR / '08_cv_por_fold.csv',
    DATA_DIR / 'features_tbx11k.csv',
]
for archivo in ARCHIVOS:
    if not archivo.exists():
        raise FileNotFoundError(f'No se encontró {archivo}. Ejecuta primero la cadena 05–09.')

modelo = joblib.load(MODELS_DIR / '08_xgboost.joblib')
selector = modelo.named_steps['selector']
clf = modelo.named_steps['clf']
scaler = modelo.named_steps.get('scaler', None)

print('Pipeline :', [paso[0] for paso in modelo.steps])
print('Selector :', type(selector).__name__)
print('Clasificador:', type(clf).__name__)
print('  classes:', clf.classes_)
print('  objective:', clf.get_params().get('objective'))
print('  feature_importances_:', clf.feature_importances_.shape)
print('  scaler:', type(scaler).__name__ if scaler is not None else 'None')

with open(MODELS_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)
print()
print('Modelo seleccionado en 08:', manifest['seleccionado_cv'])
print('Selector en 08:', manifest['selector'])
if manifest['seleccionado_cv'] != 'XGBoost':
    print('ADVERTENCIA: el manifiesto no corresponde a XGBoost; revisar el 08.')

# Nombres de las features seleccionadas en el orden de columnas del selector.
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]
sel_idx = selector.get_support(indices=True)
pipeline_names = [feat_cols[i] for i in sel_idx]

with open(DATA_DIR / '06_selected_feature_names.json', encoding='utf-8') as f:
    names_06 = json.load(f)
mismo_conjunto = set(pipeline_names) == set(names_06)
if not mismo_conjunto:
    print('ADVERTENCIA: el conjunto de features del pipeline difiere del 06; '
          'se usan las del pipeline final para explicar el modelo.')
print()
print(f'Features seleccionadas: {len(pipeline_names)}')
estado = "sí" if mismo_conjunto else "no"
print(f'Mismo conjunto que 06: {estado} '
      '(el 06 las guarda en orden de importancia; aquí en orden de columna).')
```

```
X_full = np.load(DATA_DIR / 'X_full.npy')
y_full = np.load(DATA_DIR / 'y_full.npy')
oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
scores = np.load(DATA_DIR / '08_oof_scores.npz')
fold_params = pd.read_csv(DATA_DIR / '08_cv_por_fold.csv')

if not np.array_equal(oof['y_true'].to_numpy(), y_full):
    raise ValueError('y_true del OOF no coincide con y_full.')

print()
print(f'X_full: {X_full.shape} | y_full: {y_full.shape}')
print(f'OOF XGBoost: BalAcc={balanced_accuracy_score(y_full, oof["XGBoost"]):.4f} | '
      f'Recall tb={recall_score(y_full, oof["XGBoost"], labels=[2], average=None)[0]:.4f}')
print('Parámetros XGBoost por fold:')
print(fold_params[fold_params['Modelo'] == 'XGBoost'][['Fold', 'Mejores
params']].to_string(index=False))
```

```
Pipeline : ['selector', 'clf']
Selector : SelectKBest
Clasificador: XGBClassifier
  classes: [0 1 2]
  objective: multi:softprob
  feature_importances_: (2000,)
  scaler: None

Modelo seleccionado en 08: XGBoost
Selector en 08: {'metodo': 'MI', 'n_features': 2000}
ADVERTENCIA: el conjunto de features del pipeline difiere del 06; se usan las del pipeline final para
explicar el modelo.

Features seleccionadas: 2000
Mismo conjunto que 06: no (el 06 las guarda en orden de importancia; aquí en orden de columna).

X_full: (8256, 3300) | y_full: (8256,)
OOF XGBoost: BalAcc=0.7730 | Recall tb=0.5675
Parámetros XGBoost por fold:
 Fold                                  Mejores params
    1 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    2 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    3 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    4 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
    5 {"clf__learning_rate": 0.1, "clf__max_depth": 2}
```

# 2. Preparación del espacio de interpretación

In [8]:

```
# Muestra estratificada para SHAP y agrupación de features por familia.
# TreeExplainer no necesita escalado en este pipeline: el XGBoost final no tiene scaler.

N_POR_CLASE = 500
rng = np.random.RandomState(RANDOM_STATE)
idx = []
for clase in np.unique(y_full):
    idx_clase = np.where(y_full == clase)[0]
    idx.extend(rng.choice(idx_clase, size=N_POR_CLASE, replace=False))
idx = np.array(idx)

X_sample = X_full[idx]
y_sample = y_full[idx]
X_sample_sel = selector.transform(X_sample)

feature_names = pipeline_names


def grupo_feature(nombre):
    if nombre.startswith('hog_'):
        return 'HOG'
    if nombre.startswith('phog_'):
        return 'PHOG'
    if nombre.startswith('gabor_'):
        return 'Gabor'
    if nombre.startswith('lbp_'):
        return 'LBP'
```

```
    if nombre.startswith('glcm_'):
        return 'GLCM'
    if nombre in ('mean', 'std', 'contrast', 'entropy'):
        return 'Intensidad global'
    if nombre[:2] in ('SI', 'SD', 'MI', 'MD', 'II', 'ID'):
        return 'Intensidad regional'
    return 'Otra'


GRUPOS = ['HOG', 'PHOG', 'GLCM', 'LBP', 'Gabor', 'Intensidad global', 'Intensidad regional']
grupos_por_feature = [grupo_feature(n) for n in feature_names]
conteo_grupos = pd.Series(grupos_por_feature).value_counts().reindex(GRUPOS).fillna(0).astype(int)

print('Composición de las features seleccionadas:')
print(conteo_grupos.to_string())

explainer = shap.TreeExplainer(clf)
sv = explainer(X_sample_sel)

print()
print(f'SHAP values: {sv.values.shape} (muestras, features, clases)')
print(f'base_values: {sv.base_values.shape}')
print('Eje de clase:', dict(enumerate(CLASS_NAMES)))
print('Salida explicada: margen/log-odds del XGBoost (multi:softprob), no probabilidad.')
```

```
Composición de las features seleccionadas:
HOG                   1050
PHOG                   856
GLCM                    10
LBP                     26
Gabor                   31
Intensidad global        4
Intensidad regional     23

SHAP values: (1500, 2000, 3) (muestras, features, clases)
base_values: (1500, 3)
Eje de clase: {0: 'health', 1: 'sick', 2: 'tb'}
Salida explicada: margen/log-odds del XGBoost (multi:softprob), no probabilidad.
```

# 3. SHAP global

In [9]:

```
# Importancia global: media de |SHAP| sobre muestras y clases.
# Control de coherencia con la importancia nativa del XGBoost (gain).
# No es una validación independiente: ambos describen el mismo modelo.

abs_sv = np.abs(sv.values)
imp_global = abs_sv.mean(axis=(0, 2))
top = np.argsort(imp_global)[::-1][:20]

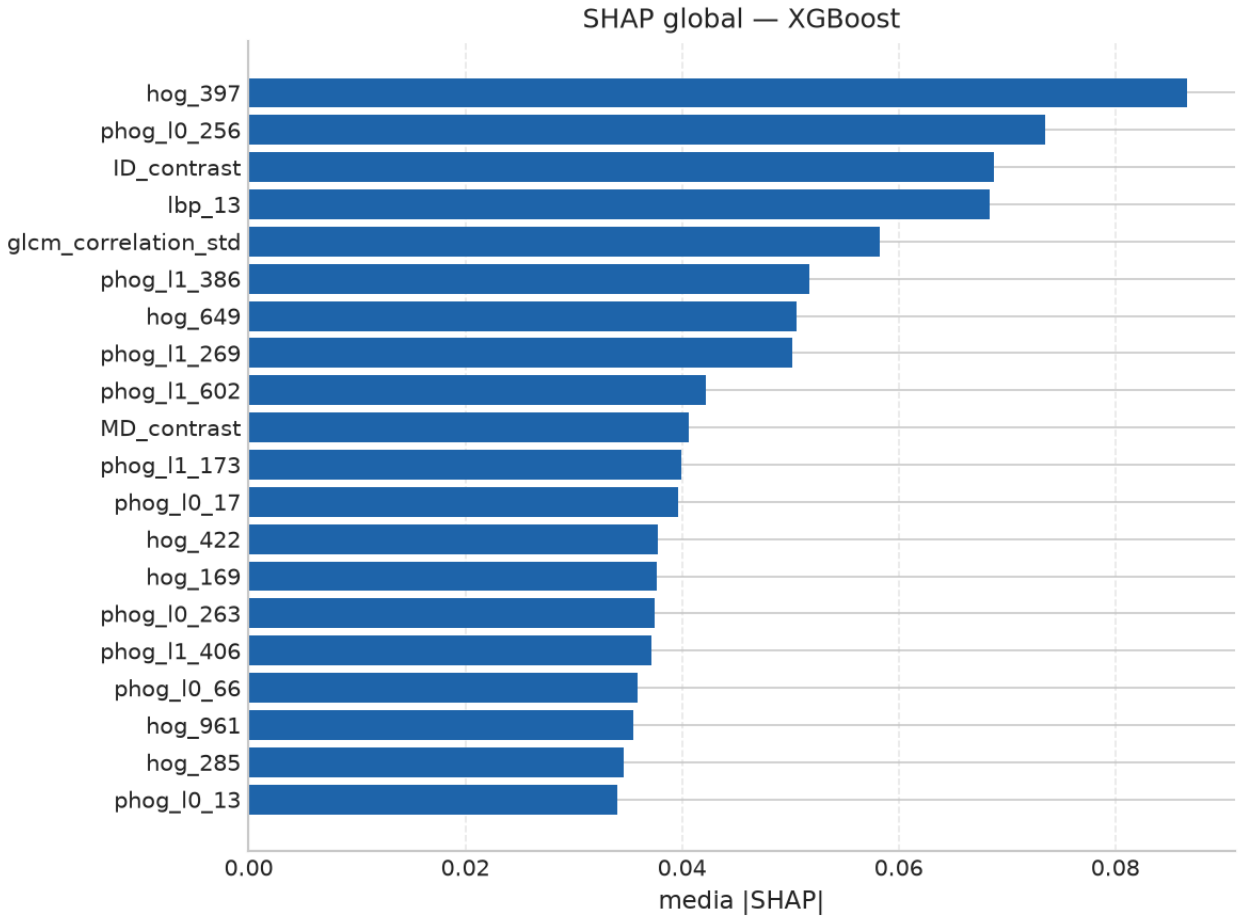
tabla = pd.DataFrame({
    'feature': [feature_names[i] for i in top],
    'mean_abs_shap': imp_global[top],
    'grupo': [grupo_feature(feature_names[i]) for i in top],
})
print('Top-20 features por |SHAP| global:')
print(tabla.round(6).to_string(index=False))

top_shap = set(top)
top_gain = set(np.argsort(clf.feature_importances_)[::-1][:20])
print()
print(f'Solapamiento top-20 SHAP / gain: {len(top_shap & top_gain)}/20')

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(tabla['feature'][::-1], tabla['mean_abs_shap'][::-1], color='#2166AC')
ax.set_xlabel('media |SHAP|', fontsize=11)
ax.set_title('SHAP global — XGBoost', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Top-20 features por |SHAP| global:
             feature  mean_abs_shap               grupo
             hog_397       0.086673                 HOG
         phog_l0_256       0.073488                PHOG
         ID_contrast       0.068790 Intensidad regional
              lbp_13       0.068362                 LBP
glcm_correlation_std       0.058229                GLCM
         phog_l1_386       0.051779                PHOG
             hog_649       0.050621                 HOG
         phog_l1_269       0.050208                PHOG
         phog_l1_602       0.042229                PHOG
         MD_contrast       0.040641 Intensidad regional
         phog_l1_173       0.039955                PHOG
          phog_l0_17       0.039684                PHOG
             hog_422       0.037746                 HOG
             hog_169       0.037642                 HOG
         phog_l0_263       0.037499                PHOG
         phog_l1_406       0.037200                PHOG
          phog_l0_66       0.035882                PHOG
             hog_961       0.035469                 HOG
             hog_285       0.034632                 HOG
          phog_l0_13       0.034043                PHOG

Solapamiento top-20 SHAP / gain: 4/20
```




# 4. SHAP por clase

In [10]:

```python
# Importancia por clase: media de |SHAP| sobre las muestras para cada clase.
# Permite comparar qué features sostienen health, sick y tb.

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, c in zip(axes, range(len(CLASS_NAMES))):
    imp = abs_sv[:, :, c].mean(axis=0)
    top_c = np.argsort(imp)[::-1][:15]

    tabla_c = pd.DataFrame({
        'feature': [feature_names[i] for i in top_c],
        'mean_abs_shap': imp[top_c],
```

```
        'grupo': [grupo_feature(feature_names[i]) for i in top_c],
    })
    print()
    print(f'--- {CLASS_NAMES[c]} ---')
    print(tabla_c.round(6).to_string(index=False))

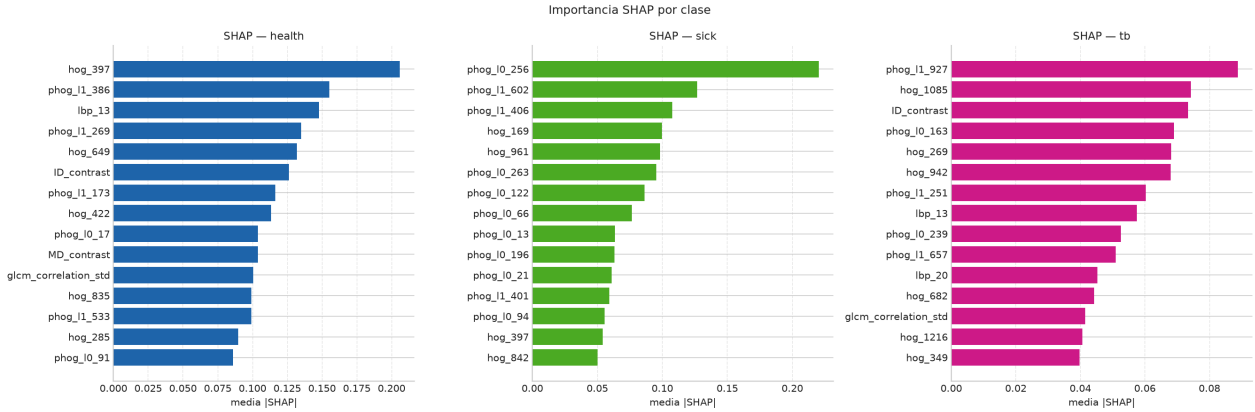
    ax.barh(tabla_c['feature'][::-1], tabla_c['mean_abs_shap'][::-1],
            color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[CLASS_NAMES[c]])
    ax.set_title(f'SHAP — {CLASS_NAMES[c]}', fontsize=11)
    ax.set_xlabel('media |SHAP|', fontsize=10)
    ax.grid(True, axis='x', linestyle='--', alpha=0.4)

fig.suptitle('Importancia SHAP por clase', fontsize=12)
fig.tight_layout()
plt.show()
```

```
--- health ---
             feature  mean_abs_shap               grupo
             hog_397       0.206067                 HOG
         phog_l1_386       0.155338                PHOG
              lbp_13       0.147644                 LBP
         phog_l1_269       0.134932                PHOG
             hog_649       0.131932                 HOG
         ID_contrast       0.126171 Intensidad regional
         phog_l1_173       0.116303                PHOG
             hog_422       0.113237                 HOG
          phog_l0_17       0.104049                PHOG
         MD_contrast       0.103759 Intensidad regional
glcm_correlation_std       0.100557                GLCM
             hog_835       0.099315                 HOG
         phog_l1_533       0.099162                PHOG
             hog_285       0.089625                 HOG
          phog_l0_91       0.086124                PHOG

--- sick ---
    feature  mean_abs_shap grupo
phog_l0_256       0.220464  PHOG
phog_l1_602       0.126688  PHOG
phog_l1_406       0.107664  PHOG
    hog_169       0.099549   HOG
    hog_961       0.098197   HOG
phog_l0_263       0.095485  PHOG
phog_l0_122       0.086195  PHOG
 phog_l0_66       0.076547  PHOG
 phog_l0_13       0.063601  PHOG
phog_l0_196       0.063088  PHOG
 phog_l0_21       0.060937  PHOG
phog_l1_401       0.059225  PHOG
 phog_l0_94       0.055719  PHOG
    hog_397       0.053950   HOG
    hog_842       0.050224   HOG

--- tb ---
             feature  mean_abs_shap               grupo
         phog_l1_927       0.088800                PHOG
            hog_1085       0.074202                 HOG
         ID_contrast       0.073426 Intensidad regional
         phog_l0_163       0.068948                PHOG
             hog_269       0.068169                 HOG
             hog_942       0.067995                 HOG
         phog_l1_251       0.060199                PHOG
              lbp_13       0.057442                 LBP
         phog_l0_239       0.052499                PHOG
         phog_l1_657       0.050987                PHOG
              lbp_20       0.045273                 LBP
             hog_682       0.044257                 HOG
glcm_correlation_std       0.041550                GLCM
            hog_1216       0.040564                 HOG
             hog_349       0.039786                 HOG
```




# 5. Importancia por familia de características

In [11]:

```
# Media de |SHAP| por feature dentro de cada familia, para comparar familias con
# distinto número de columnas sin que las más numerosas dominen por conteo.

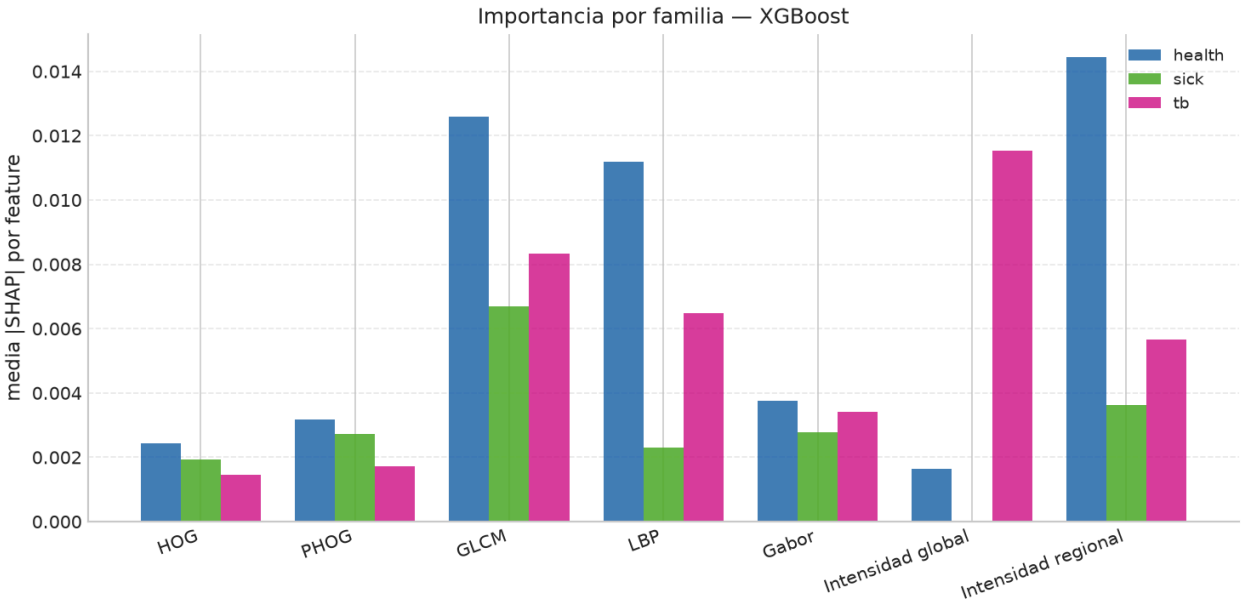
def importancia_grupo(imp_vec):
    filas = []
    for grupo in GRUPOS:
        idx_grupo = [i for i, g in enumerate(grupos_por_feature) if g == grupo]
        if not idx_grupo:
            continue
        filas.append({
            'Grupo': grupo,
            'n_features': len(idx_grupo),
            '%_features': 100 * len(idx_grupo) / len(feature_names),
            'media_por_feature': float(np.mean(imp_vec[idx_grupo])),
            'total_grupo': float(np.sum(imp_vec[idx_grupo])),
        })
    return pd.DataFrame(filas)


tablas = []
for c, clase in enumerate(CLASS_NAMES):
    df_clase = importancia_grupo(abs_sv[:, :, c].mean(axis=0))
    df_clase['Clase'] = clase
    tablas.append(df_clase)

tabla_grupos = pd.concat(tablas, ignore_index=True)
print('Importancia media por feature y familia:')
print(tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature')
      .reindex(GRUPOS).round(6).to_string())

pivote = tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature').reindex(GRUPOS)
x = np.arange(len(pivote.index))
ancho = 0.26
fig, ax = plt.subplots(figsize=(10, 5))
for k, clase in enumerate(CLASS_NAMES):
    ax.bar(x + (k - 1) * ancho, pivote[clase], ancho, label=clase,
           color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[clase],
           alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(pivote.index, rotation=20, ha='right')
ax.set_ylabel('media |SHAP| por feature', fontsize=11)
ax.set_title('Importancia por familia — XGBoost', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, axis='y', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Importancia media por feature y familia:
Clase                 health      sick        tb
Grupo
HOG                 0.002435  0.001928  0.001451
PHOG                0.003166  0.002730  0.001715
GLCM                0.012598  0.006693  0.008322
LBP                 0.011194  0.002298  0.006488
Gabor               0.003754  0.002786  0.003407
Intensidad global   0.001653  0.000000  0.011523
Intensidad regional 0.014436  0.003631  0.005669
```




# 6. Explicaciones individuales OOF

In [12]:

```python
# Para explicar un error OOF se reconstruye el modelo del fold que generó esa predicción:
# mismo selector MI, mismos parámetros y mismo sample_weight balanceado que el 08.

with open(DATA_DIR / '06_seleccion.json', encoding='utf-8') as f:
    seleccion = json.load(f)
METHOD_SEL = seleccion['metodo']
N_SEL = int(seleccion['n_features'])
print(f'Selector del 06: {METHOD_SEL}, N={N_SEL}')

params_por_fold = {
    int(fila['Fold']): json.loads(fila['Mejores params'])
    for _, fila in fold_params[fold_params['Modelo'] == 'XGBoost'].iterrows()
}

cv_outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
folds = list(cv_outer.split(X_full, y_full))
fold_of_index = {}
for fold, (_, va_idx) in enumerate(folds, start=1):
    for i in va_idx:
        fold_of_index[int(i)] = fold

fold_cache = {}


def obtener_modelo_fold(fold):
    """Reconstruye el XGBoost del fold con selector MI y sample_weight balanceado."""
    if fold in fold_cache:
        return fold_cache[fold]

    tr_idx, va_idx = folds[fold - 1]
    params = params_por_fold[fold]

    selector_f = build_selector(METHOD_SEL, N_SEL, random_state=RANDOM_STATE, n_jobs=-1)
    selector_f.fit(X_full[tr_idx], y_full[tr_idx])
    X_tr = selector_f.transform(X_full[tr_idx])

    clf_f = XGBClassifier(
```

```
        n_estimators=300, max_depth=params.get('clf__max_depth', 6),
        learning_rate=params.get('clf__learning_rate', 0.1),
        subsample=0.8, colsample_bytree=0.8,
        objective='multi:softprob', num_class=len(CLASS_NAMES),
        eval_metric='mlogloss', tree_method='hist',
        n_jobs=1, random_state=RANDOM_STATE,
    )
    clf_f.fit(X_tr, y_full[tr_idx],
              sample_weight=compute_sample_weight('balanced', y_full[tr_idx]))

    fold_cache[fold] = (selector_f, clf_f, va_idx)
    return fold_cache[fold]


y_true = oof['y_true'].to_numpy()
pred_xgb = oof['XGBoost'].to_numpy()
casos = {
    'tb correcto': np.where((y_true == 2) & (pred_xgb == 2))[0],
    'tb falso negativo': np.where((y_true == 2) & (pred_xgb != 2))[0],
    'tb falso positivo': np.where((y_true != 2) & (pred_xgb == 2))[0],
}
seleccion_casos = {nombre: int(rng.choice(indices)) for nombre, indices in casos.items()}

for etiqueta, idx_ejemplo in seleccion_casos.items():
    fold = fold_of_index[idx_ejemplo]
    selector_f, clf_f, va_idx = obtener_modelo_fold(fold)

    x_sel = selector_f.transform(X_full[[idx_ejemplo]])
    pred_fold = int(clf_f.predict(x_sel)[0])
    nombres_fold = [feat_cols[i] for i in selector_f.get_support(indices=True)]
    explainer_f = shap.TreeExplainer(clf_f)
    sv_i = explainer_f(x_sel)

    print('=' * 72)
    print(f'{etiqueta} | índice={idx_ejemplo} | fold={fold}')
    print(f'  real={CLASS_NAMES[int(y_true[idx_ejemplo])]} | '
          f'OOF={CLASS_NAMES[int(pred_xgb[idx_ejemplo])]} | fold={CLASS_NAMES[pred_fold]}')
    if pred_fold != int(pred_xgb[idx_ejemplo]):
        print('  ADVERTENCIA: la predicción reconstruida no coincide con la OOF.')

    for clase in {int(y_true[idx_ejemplo]), int(pred_xgb[idx_ejemplo])}:
        contrib = sv_i.values[0, :, clase]
        top_ej = np.argsort(np.abs(contrib))[::-1][:8]
        tabla_ej = pd.DataFrame({
            'feature': [nombres_fold[i] for i in top_ej],
            'shap': contrib[top_ej],
            'valor_feature': x_sel[0, top_ej],
        })
        print(f'  Clase {CLASS_NAMES[clase]}:')
        print(tabla_ej.round(4).to_string(index=False))
```

```
Selector del 06: MI, N=2000
================================================================================
tb correcto | índice=7470 | fold=3
  real=tb | OOF=tb | fold=tb
  Clase tb:
    feature    shap  valor_feature
ID_contrast  0.3589       245.0000
     lbp_20  0.2369         0.0066
     lbp_21  0.1691         0.0041
    hog_302 -0.1568         0.2607
    hog_847  0.1009         0.2741
   hog_1216  0.0886         0.0502
    hog_647  0.0836         0.2585
   hog_1085 -0.0772         0.0749
================================================================================
tb falso negativo | índice=7637 | fold=3
  real=tb | OOF=sick | fold=sick
  Clase sick:
    feature    shap  valor_feature
phog_l1_406  0.3023         0.2268
phog_l0_256  0.2297         0.2405
   hog_1048  0.2115         0.3221
phog_l1_956  0.2069         0.0979
    hog_961 -0.1358         0.2673
phog_l0_196  0.0999         0.3603
    hog_910  0.0987         0.2954
phog_l1_481  0.0965         0.1387
  Clase tb:
            feature    shap  valor_feature
        ID_contrast  0.1979       246.0000
        phog_l1_657  0.1407         0.4309
        phog_l1_956 -0.1204         0.0979
gabor_f0.1_t45_mean -0.1063         0.8946
        phog_l0_239 -0.0920         0.4231
            hog_647  0.0836         0.3257
       phog_l1_1045  0.0704         0.2695
        phog_l0_163  0.0674         0.1688
================================================================================
tb falso positivo | índice=1083 | fold=3
  real=health | OOF=tb | fold=tb
  Clase health:
             feature    shap  valor_feature
glcm_correlation_std -0.4718         0.0346
         phog_l1_386 -0.2239         0.0000
         phog_l1_269 -0.2223         0.0000
             hog_397 -0.2136         0.0000
             hog_649 -0.2039         0.0000
  gabor_f0.1_t90_std -0.1693         1.2717
        SD_contrast -0.1657         0.0000
             hog_422 -0.1492         0.0000
  Clase tb:
            feature    shap  valor_feature
  glcm_contrast_std  0.3766         0.3913
            lbp_13  0.1569         0.0007
 gabor_f0.2_t0_mean -0.1138         0.0699
       phog_l0_163  0.0730         0.0000
            hog_884 -0.0708         0.4273
             SI_std  0.0621         0.0000
gabor_f0.2_t135_std  0.0552         0.4564
       phog_l1_622 -0.0544         0.1932
```

# 7. Atribución espacial y cajas de lesión

In [13]:

```python
# Mapas de oclusión 8x8 sobre el XGBoost final: se tapa cada celda, se reextrae el
# vector de 3300 features y se mide la caída de la probabilidad de TB.
# Se comparan con las cajas de lesión anotadas de TBX11K.

coco_path = Path('..') / 'data' / 'TBX11K' / 'annotations' / 'json' / 'TBX11K_trainval.json'
if not coco_path.exists():
    raise FileNotFoundError(f'No se encontró {coco_path}')

with open(coco_path, encoding='utf-8') as f:
```

```
    coco = json.load(f)

meta = pd.read_csv(DATA_DIR / 'full_meta.csv')
basename_to_idx = {Path(p).name: i for i, p in enumerate(meta['image_path'])}
imagenes_coco = {img['id']: img['file_name'] for img in coco['images']}
cat_tb = [c['id'] for c in coco['categories']
          if c['name'].lower() in ('tb', 'tuberculosis')]
cat_tb = cat_tb[0] if cat_tb else 2

cajas = {}
for ann in coco['annotations']:
    if ann.get('category_id') != cat_tb:
        continue
    file_name = imagenes_coco.get(ann['image_id'])
    if file_name is None:
        continue
    idx = basename_to_idx.get(Path(file_name).name)
    if idx is None:
        continue
    path = meta.loc[idx, 'image_path']
    cajas.setdefault(path, []).append(tuple(float(v) for v in ann['bbox']))

mask_tb = (oof['y_true'] == 2) & (oof['XGBoost'] == 2)
paths_tb_ok = set(meta.loc[mask_tb.to_numpy(), 'image_path'])
candidatos = sorted(set(cajas) & paths_tb_ok)
print(f'Imágenes de TB con cajas: {len(cajas)} | TB correctas con caja: {len(candidatos)}')
if not candidatos:
    raise ValueError('No hay imágenes de TB correctas con caja.')

ejemplos = candidatos[:3]
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]

GRID = 8
fig, axes = plt.subplots(1, len(ejemplos), figsize=(5.5 * len(ejemplos), 5))
if len(ejemplos) == 1:
    axes = [axes]

for ax, path in zip(axes, ejemplos):
    idx = int(meta.index[meta['image_path'] == path][0])
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        continue

    score_base = clf.predict_proba(selector.transform(X_full[[idx]]))[0, 2]
    heatmap = np.zeros((GRID, GRID), dtype=float)
    h, w = img.shape
    cell_h, cell_w = h // GRID, w // GRID

    for gy in range(GRID):
        for gx in range(GRID):
            occ = img.copy()
            y0, y1 = gy * cell_h, (gy + 1) * cell_h
            x0, x1 = gx * cell_w, (gx + 1) * cell_w
            occ[y0:y1, x0:x1] = 0

            with tempfile.NamedTemporaryFile(suffix='.png', delete=False) as tmp:
                tmp_path = tmp.name
            cv2.imwrite(tmp_path, occ)
            try:
                f = extract_features(tmp_path, 2, 'tb')
                vec = np.array([f[k] for k in feat_cols], dtype=np.float32).reshape(1, -1)
                score = clf.predict_proba(selector.transform(vec))[0, 2]
                heatmap[gy, gx] = score_base - score
            finally:
                Path(tmp_path).unlink(missing_ok=True)

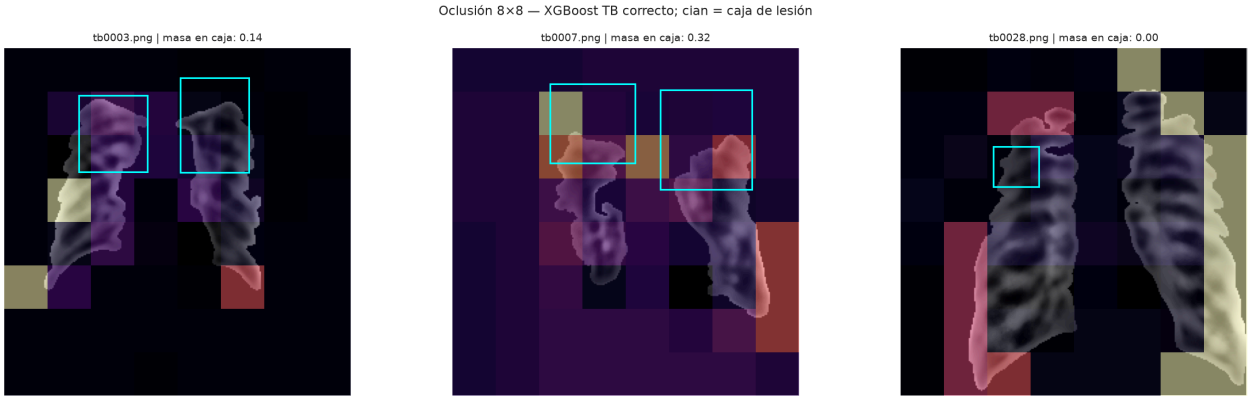
    cajas_img = cajas[path]
    inside = total = 0.0
    for gy in range(GRID):
        for gx in range(GRID):
            drop = max(heatmap[gy, gx], 0.0)
            total += drop
            cx = gx * cell_w + cell_w / 2
            cy = gy * cell_h + cell_h / 2
```

```
            if any(x <= cx <= x + bw and y <= cy <= y + bh
                   for x, y, bw, bh in cajas_img):
                inside += drop
    fraccion = inside / total if total > 0 else float('nan')

    img_256 = preprocess(path)
    ax.imshow(img_256, cmap='gray')
    ax.imshow(heatmap, extent=[0, 256, 256, 0], cmap='magma', alpha=0.55)
    for x, y, bw, bh in cajas_img:
        ax.add_patch(Rectangle((x * 0.5, y * 0.5), bw * 0.5, bh * 0.5,
                               fill=False, edgecolor='cyan', linewidth=1.5))
    ax.set_title(f'{Path(path).name} | masa en caja: {fraccion:.2f}', fontsize=9)
    ax.axis('off')

fig.suptitle('Oclusión 8×8 — XGBoost TB correcto; cian = caja de lesión', fontsize=11)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '10_oclusion_lesiones.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Imágenes de TB con cajas: 169 | TB correctas con caja: 97
```




# 8. Limitaciones y discusión

- Los valores SHAP son atribuciones al **output del XGBoost** (margen/log-odds), no probabilidades; el eje de clase es `clf.classes_`.
- SHAP y `feature_importances_` son complementarios, no validaciones independientes.
- HOG/PHOG están correlacionadas; SHAP reparte la atribución y no debe leerse como causalidad.
- La segmentación pulmonar es heurística; una máscara imperfecta puede atenuar la señal.
- Las explicaciones individuales usan el modelo del fold que produjo la predicción OOF.
- Sin validación externa ni IDs de paciente, la interpretación es descriptiva.

# Conclusiones

El notebook explica el XGBoost final con SHAP por clase y por familia, y revisa ejemplos OOF con el modelo del fold correspondiente. Las conclusiones sustantivas deben redactarse con las tablas y figuras de las secciones 3–7.

Las métricas de rendimiento no se recalculan aquí: pertenecen al notebook 09.

**Fin del pipeline de modelado, evaluación e interpretabilidad.**